

Práctica 3.2: Programación de OMP en Python

Índice

Objetivos	1
Introducción a PyOMP	1
Instalación	1
Uso en “Aulas Virtuales FDI Linux VGPU”	2
Ejercicio 1: Hello World	3
Ejercicio 2: Explotación de Hilos con OpenMP	3
Ejercicio 3: Explotación de GPU con OpenMP	5
Ejercicio 4: Kmeans explotación multihilo en CPU (entrega)	5
Ejercicio 5: Multiplicación matrices en GPU (entrega)	6
Resultado esperado	6

Objetivos

- Familiarizarse con PyOMP
- Relacionar OpenMP con PyOMP así como la correlación entre las directivas `parallel`, `for`, `task` y `target`
- Evaluar el rendimiento de las soluciones paralelas frente a las versiones secuenciales

Introducción a PyOMP

PyOMP es una interfaz que permite integrar la paralelización mediante OpenMP directamente desde Python, ofreciendo al desarrollador una herramienta sencilla pero potente para acelerar la ejecución de código utilizando múltiples hilos del procesador. A través de esta librería, es posible paralelizar tareas intensivas en cómputo, como operaciones matriciales y ciclos repetitivos, aprovechando efectivamente los recursos disponibles en CPU o GPU.

Una de las grandes ventajas de PyOMP es que facilita la implementación de instrucciones paralelas en código Python, que habitualmente es secuencial. Esto permite obtener importantes ganancias en rendimiento sin tener que recurrir a lenguajes tradicionalmente usados para alto rendimiento, como C o Fortran. En combinación con herramientas como Numba, PyOMP puede generar códigos numéricos optimizados, manteniendo la simplicidad y legibilidad del código Python original.

Instalación

- La instalación está soportada a través de entornos conda como viene descrito en el repositorio. Si en el equipo de laboratorio no estuviese instalado conda, se puede descargar:

```
user@host% wget https://repo.anaconda.com/archive/Anaconda3-2025.12-2-Linux-x86_64.sh
user@host% bash ./Anaconda3-2025.12-2-Linux-x86_64.sh
yes
user@host% source ~/.bashrc
```

Vamos a crear un entorno con toda la paquetería que se va a usar en PyOMP. Para ello emplearemos el plano de construcción contenido en el fichero `environmentPyOMP.yml` que se encuentra en los el ficheros `p3-2.tar.gz` de la práctica. Dicho fichero contiene el nombre del entorno: `pyomp_psd`; los canales donde bajará encontrará los paquetes a descargar, y las dependencias de la paquetería:

```
# Crear entorno
user@host% conda env create -f environmentPyOMP.yml
.....
done
#
# To activate this environment, use
#
#     $ conda activate pyomp_psd
#
# To deactivate an active environment, use
#
#     $ conda deactivate

# Activación
user@host% conda activate pyomp_psd
```

Tiene soporte para las GPUs de NVIDIA desde la “CUDA capability” `sm_50` (GeForce GTX 10xx) hasta `sm_86` (Geforce RTX30xx). También tiene soporte para alguna GPUs de AMD (ej: AMD Radeon RX 6700)

Uso en “Aulas Virtuales FDI Linux VGPU”

Se puede hacer funcionar el entorno de las Aulas Virtuales UCM cargando el entorno “**FDI LINUX VGPU**”.

Una vez en el entorno se puede replicar las instrucciones anteriores:

1. Descargar Anaconda e instalarlo `bash ./Anaconda3-2025.12-2-Linux-x86_64.sh`
2. Crear el entorno virtual mediante el fichero de configuración **environmentPyOMP.yml** cambiando la prioridad a flexible temporalmente:

```
(base) usufdi@C-117-L-VGPU-04:~/Descargas$ conda config --set channel_priority flexible
(base) usufdi@C-117-L-VGPU-04:~/Descargas$ conda env create -f enviromentPyOMP.yml
(base) usufdi@C-117-L-VGPU-04:~/Descargas$ conda activate pyomp_psd
```

3. Se puede testear tanto el uso de CPU como de GPU:

```
(pyomp_psd) usufdi@C-117-L-VGPU-04:~/Descargas$ OMP_NUM_THREADS=2 python3 hello.py
hello
world
hello
world
DONE

(pyomp_psd) usufdi@C-117-L-VGPU-04:~/Descargas$ nsys nvprof python3 axpy.py
WARNING: python3 and any of its children processes will be profiled.

Collecting data...
[3. 3. 3. 3. 3. 3. 3. 3. 3. 3.]
[3. 3. 3. 3. 3. 3. 3. 3. 3. 3.]
Generating '/tmp/nsys-report-6ab4.qdstrm'
....
[5/7] Executing 'cuda_gpu_kern_sum' stats report

Time (%)   Total Time (ns)   Instances   Avg (ns)   Med (ns)   Min (ns)   Max (ns)   StdDev (ns)
```

```
100,0      3.680      1      3.680,0      3.680,0      3.680      3.680      0,0      __main__::dev
...
```

Ejercicio 1: Hello World

El primer ejemplo “hello.py” utiliza la librería Numba que permite acelerar la ejecución de funciones numéricas, mediante la compilación de código Python a código máquina optimizado con el mecanismo Just-in-Time, o JIT, lo que a su vez el estándar de compilación del LLVM permitiendo la interacción de Python con lenguajes compilados como C/C++ o Fortran.

Es importante destacar en el código el decorador `@njit` que permite compilar la función `hello()` usando Numba, mejorando su rendimiento. Además este aspecto permite la utilización de librerías de paralelización como las estudiadas en la asignatura como OpenMP con el decorador `with openmp("parallel")` : que inicia una región paralela utilizando OpenMP. Esto significa que las instrucciones dentro de este bloque pueden ejecutarse simultáneamente por múltiples hilos.

```
from numba import njit
from numba.openmp import openmp_context as openmp

@njit
def hello():
    with openmp("parallel"):
        print("hello")
        print("world")
hello()
print("DONE")
```

Al igual que en la práctica de OpenMP, se puede controlar el número de hilos con la variable de entorno `OMP_NUM_THREADS` como se puede observar en la siguiente ejecución:

```
(PyOMP) user@host% OMP_NUM_THREADS=2 python3 hello.py
hello
world
hello
world
Done!!
```

Ejercicio 2: Explotación de Hilos con OpenMP

- Vamos a tratar el ejemplo de cálculo de π como aproximación de la integral:

$$\pi = \int_0^1 \frac{4}{1+x^2} \delta x$$

- Podemos aproximar la integral como la suma de los rectángulos
 - donde cada rectángulo tiene de ancho Δx y la altura $F(x_i)$

$$\pi \simeq \sum_N^{i=0} F(x_i) \Delta x$$

La versión secuencial recogida en el código `pi_serie.py` es el punto de partida más sencillo para calcular π mediante integración numérica usando la regla del punto medio. Este algoritmo consiste en sumar áreas de rectángulos bajo una curva determinada. En Python, esta implementación es simple y utiliza un único ciclo iterativo, lo cual limita su velocidad al uso de un solo núcleo del procesador.

```
def piFunc(NumSteps):
    step=1.0/NumSteps
    sum = 0.0
    x = 0.5
    for i in range(NumSteps):
```

```

        x+=step
        sum += 4.0/(1.0+x*x)
    pi=step*sum
    return pi

```

La versión de OpenMP que explota la distribución de iteraciones entre los hilos OpenMP recogida en el código `pi_omp_loop.py`, emplea la directiva `parallel for` implementando una reducción eficiente de la variable `sum`:

```

@njit
def piFunc(NumSteps):
    step = 1.0/NumSteps
    sum = 0.0
    with openmp("parallel for private(x) reduction(+:sum)":
        for i in range(NumSteps):
            x = (i+0.5)*step
            sum += 4.0/(1.0 + x*x)
    pi=step*sum
    return pi

```

El código `pi_omp_spmc.py` explota el paralelismo SPMD (Single Program Multiple Data) distribuyendo manualmente bloques de iteraciones entre diferentes hilos. Cada hilo realiza su suma parcial, almacenando los resultados en un arreglo que luego se combina para obtener el valor final. Esta estrategia permite control explícito sobre cómo se asignan las iteraciones a cada hilo.

```

@njit
def piFunc(NumSteps):
    step = 1.0/NumSteps
    partialSums = np.zeros(NTHREADS)
    with openmp("parallel shared(partialSums) private(threadID,numThrds,i,x,localSum) num_threads(8)":
        threadID = omp_get_thread_num()
        numThrds = omp_get_num_threads()
        localSum = 0.0
        for i in range(threadID, NumSteps, numThrds):
            x = (i+0.5)*step
            localSum = localSum + 4.0/(1.0 + x*x)
        partialSums[threadID] = localSum
    return step*np.sum(partialSums)

```

Por último la versión de tareas o `tasks` del fichero `pi_omp_tasks.py` divide recursivamente el problema en subproblemas más pequeños. Cada tarea se asigna dinámicamente a los hilos disponibles, facilitando la gestión dinámica del balance de carga, especialmente útil en sistemas heterogéneos o cuando el número de iteraciones es muy grande.

```

MIN_BLK = 1024
@njit
def piComp(Nstart, Nfinish, step):
    iblk = Nfinish-Nstart
    if(iblk<MIN_BLK):
        sum = 0.0
        for i in range(Nstart,Nfinish):
            x= (i+0.5)*step
            sum += 4.0/(1.0 + x*x)
    else:
        sum1 = 0.0
        sum2 = 0.0
        with openmp ("task shared(sum1)":
            sum1 = piComp(Nstart, Nfinish-iblk/2,step)
        with openmp ("task shared(sum2)":
            sum2 = piComp(Nfinish-iblk/2,Nfinish,step)
        with openmp ("taskwait"):
            sum = sum1 + sum2

```

```

    return sum

@njit
def piFunc(NumSteps):
    step = 1.0/NumSteps
    sum = 0.0
    with openmp ("parallel"):
        with openmp ("single"):
            sum = piComp(0, NumSteps, step)
    pi = step*sum
    return step*sum

```

Ejercicio 3: Explotación de GPU con OpenMP

El ejemplo `axpy.py` utiliza la operación matemática básica conocida como AXPY (que significa “A times X Plus Y”), para ilustrar una implementación descargando un código a la GPU. Para ello al igual que en OpenMP la directiva `omp target` descarga el kernel al acelerador.

Pero antes se ha de trasladar la información necesaria a la memoria del acelerador con la cláusula `map`.

```

@njit
def axpy_gpu(x, y, a, N):
    with openmp("target map(tofrom: y) map(to: x)"):
        for i in range(N):
            y[i] = a*x[i] + y[i]

    return y

```

De igual manera que en ejercicio anterior se muestra el código relativo al cálculo de π realizando el cómputo en la GPU mediante la directiva `target` en el fichero `pi_target.py`. En este caso se pone el foco en la distribución de la carga de trabajo entre los diferentes niveles jerárquico de la GPU (SMs, CUDA-core...):

```

@njit
def piFunc(NumSteps):
    step = 1.0/NumSteps
    sum = 0.0
    start_time = omp_get_wtime()
    with openmp("target teams distribute parallel for private(x) reduction(+:sum)"):
        for i in range(NumSteps):
            x = (i+0.5)*step
            sum += 4.0/(1.0 + x*x)

    pi = step * sum
    runtime = omp_get_wtime() - start_time
    print("pi = ", pi, "runtime = ", runtime)
    return pi

```

Ejercicio 4: Kmeans explotación multihilo en CPU (entrega)

K-means es un algoritmo de aprendizaje no supervisado utilizado para resolver problemas de clustering o agrupamiento. Su objetivo es particionar un conjunto de datos en K grupos (clusters) de tal forma que los elementos dentro de un mismo grupo sean lo más similares posible entre sí, y lo más distintos posible de los elementos de otros grupos.

El funcionamiento básico del algoritmo sigue los siguientes pasos:

1. Inicialización: Se seleccionan aleatoriamente K centroides iniciales (puntos representativos de cada grupo).
2. Asignación: Cada punto del conjunto de datos se asigna al centroide más cercano (según distancia euclídea u otra métrica).

3. Actualización: Se recalculan los centroides como el promedio de los puntos asignados a cada grupo.
4. Repetición: Los pasos de asignación y actualización se repiten hasta que los centroides no cambian significativamente o se alcanza un número máximo de iteraciones.

El código `kmeans_serial.py` lee un dataset de un fichero tipo CSV, e invoca a la función `kmeans` donde para un número de iteraciones, calcula la distancias a los centroides para asignar la clase con la función `compute_distances` y actualiza los centroides con `update_centroids`.

La implementación paralela de ambas funciones tiene los siguiente retos:

- `compute_distances` ofrece una explotación de paralelismo mediante hilo evidente en el primer de los bucles `for i in range(n_samples)` puesto que calcula la distancia al centroide menor y le asigna dicha etiqueta
- `update_centroids` presenta el problema de reducción ya estudiado a lo largo de la asignatura, por un lado se acumulan los puntos asociado a cada cluster en `counts[c]` (reducción) para posteriormente calcular la nueva posición del centroide como valor medio las posiciones de los puntos del dataset que pertenecen a ese cluster `new_centroids[j, f] /= counts[j]` (reducción).

Ejercicio 5: Multiplicación matrices en GPU (entrega)

La multiplicación de matrices es una operación fundamental del álgebra lineal donde dos matrices se combinan para generar una tercera.

La multiplicación de matrices aparece constantemente en tareas de ciencia de datos y aprendizaje automático, ya que permite representar cálculos complejos de forma compacta y eficiente:

- Regresión Lineal y Logística: las predicciones se obtienen calculando una suma ponderada de las variables de entrada: $y = \Phi * X$, donde X es la matriz de características y Φ el vector de parámetros
- Redes Neuronales Convolucionales (CNNs): utilizan operaciones de convolución, que internamente se transforman en multiplicaciones de matrices.
- Reducción de Dimensionalidad por Componentes Principales (PCA): requieren calcular matrices de covarianza, autovalores y proyecciones, todos ellos basados en productos matriciales.

Debido a que la multiplicación de matrices es tan común y costosa computacionalmente, optimizar su implementación (por ejemplo, usando bibliotecas como BLAS, cuBLAS o MKL) tiene un impacto directo en la velocidad de entrenamiento e inferencia de modelos.

Resultado esperado

Al finalizar esta práctica, el alumno será capaz de:

- Comprender el funcionamiento básico de PyOMP y cómo se integra con Numba para aprovechar la paralelización en CPU y GPU desde Python
- Explotar el paralelismo de hilos con la directiva `parallel`
- Explotar el paralelismo heterogéneo en GPU mediante la directiva `target`